

Report for assignment 4

– Godot Issue Resolution –

Josefine Nyholm

Erik Olsson

Albin Blomqvist

Avid Fayaz

Chow Pun Chun

Royal Institute of Technology (KTH)

DD2480 VT26

2026-03-03

Contents

Project.....	1
Onboarding experience.....	1
Effort spent.....	2
Overview of issue(s) and work done.....	4
Requirements for the new feature or requirements affected by functionality being refactored.....	5
Code changes.....	5
Declaring the resource as a tool script.....	5
Patch.....	6
Test results.....	7
Initial results.....	7
CI-tests:.....	7
Unit tests:.....	7
UML class diagram and its description.....	8
Key changes/classes affected.....	8
Edit MRP.....	8
Run MRP:.....	8
Architectural overview.....	9
Relation to design pattern(s).....	10
Overall experience.....	11
Assessment of team using Essence standard.....	11
What are your main take-aways from this project? What did you learn?.....	11
How did you grow as a team, using the Essence standard to evaluate yourself?.....	12
How would you put your work in context with best software engineering practice?.....	12
Statement of contribution.....	13
References.....	15

Project

Name: Godot

URL: <https://github.com/godotengine/godot>

Assignment URL: <https://github.com/Soffan-Group-6/godot-4.4.1-stable>

A cross platform game engine with 2d and 3d capability.

Onboarding experience

In this assignment it was decided to use the same project as previously. Because of this the onboarding experience was the same. However, we were somewhat more familiar with the project structure, plugin framework and the Godot engine environment. The onboarding included identifying key scripts and understanding how tool scripts and resources interact in the editor.

In terms of setting up and forking the repo to contain the necessary releases for the chosen issue, two extra branches representing tag 4.4.1-stable and 4.4 were added.

Effort spent

Team member	Task	Time spent
Josefine Nyholm	Plenary discussions / meetings	~4 h
	Discussions within parts of the group	(All discussions are open for the whole group)
	Reading documentation	~3.5 h
	Configuration and setup	Forked repo ~10 min Added issue related branches ~10 min Branch setup ~2h Setup repo settings ~30 min Created related issue ~10 min
	Analyzing code/output	Testing ~18h
	Writing documentation	~16 h
	Writing code	~2h
	Running code	~5h
Erik Olsson	Plenary discussions / meetings	3h
	Discussions within parts of the group	
	Reading documentation	2h
	Configuration and setup	
	Analyzing code/output	10h
	Writing documentation	0,5h
	Writing code	2h
	Running code	3h

Team member	Task	Time spent
Albin Blomqvist	Plenary discussions / meetings	2h
	Discussions within parts of the group	
	Reading documentation	2h
	Configuration and setup	1h
	Analyzing code/output	6h
	Writing documentation	4h
	Writing code	1h
	Running code	3h
Chow Pun Chun	Plenary discussions / meetings	1h
	Discussions within parts of the group	
	Reading documentation	4h
	Configuration and setup	2h
	Analyzing code/output	6h
	Writing documentation	2h
	Writing code	1h
	Running code	1h
Avid Fayaz	Plenary discussions / meetings	~3h
	Discussions within parts of the group	
	Reading documentation	~4h
	Configuration and setup	~1h
	Analyzing code/output	~1h

Team member	Task	Time spent
	Writing documentation	~1h
	Writing code	~3h
	Running code	1h

Overview of issue(s) and work done

Title: *“All empty dictionaries in resources loaded from disk point to the same object in memory”*

URL: <https://github.com/godotengine/godot/issues/107588>

Summary

The issue concerns multiple .tres resource files containing empty Dictionary objects that are being loaded in as a single object in memory rather than individual objects. This means that when entries are added, they are added to all of the resources even if they should be handled as individual objects. Further, this bug is only present whilst editing a project similar to the provided Minimal Reproduction Project (MRP), but not when running it or when the Dictionary objects contain at least one element [1].

Scope

The issue concerns the functionality of multiple .tres resource files containing Dictionary objects being handled as a single object when modified or loaded from memory by a main screen plugin Scene. When adding elements of sorts in the resource files of type .tres, the elements show up in all resources Dictionary objects of type rather than the one modified. The functionality of the .tres files is to provide resources such as settings, widgets or in this case Resource objects containing certain attributes, that the plugin needs, in this case the main screen plugin Scene.

In terms of functionality, the MRP project and its plugin has its core in the EditorNode class [1]. The EditorNode is what handles attributes and functionality within the editor, such as importing Scene, Resource and Plugin objects. The MRP Plugin specifically contains two extended Button objects which, when pressed, should add a key to the respective Dictionary object both within the editor and when the project is running.

Furthermore, the code affected by this issue contains many layers. Since it all begins at the EditorNode it creates a long pipeline of classes, methods and objects being used. Some core classes affected, apart from the EditorNode, would be Resource, Node, GDScript, Object, ResourceLoader, Dictionary and ConfigFile. To elaborate, the class GDScript is relevant due to the plugin being implemented through GDScript files.

Reproduction guide

1. Download the MRP [1]
2. Run the godot binary
3. In the Project Manager, click “Import” and import the MRP
4. Select the MRP and click “Edit”
5. Click the “Plugin” button in the header
6. Import the scene
7. Open the scene
8. Click one of the buttons and observe the key being saved to both resource dictionaries.
9. Click “Run” and do the same thing. Notice that the key only is saved to the correct dictionary now. (“Run” can also be clicked directly in the Project Manager)

Requirements for the new feature or requirements affected by functionality being refactored

Requirement 1: Prevent shared dictionaries

Dictionaries in loaded resources must be independent and not share memory between instances. Two resources that initially contain empty dictionaries should each have their own separate dictionary. Modifying one resource’s dictionary should not affect the other.

Requirement 2: Tool script behavior

Resources loading in @tool scripts must allow their directories to be modified correctly in the editor, without being read-only or unexpectedly shared.

Trace tests to requirements.

The existing test case “[Resource] Saving and loading” in “tests/core/io/test_resources.cpp” tests saving and loading resources, but not resources with dictionaries. Thus it does not test requirement 1. The only other test that checks loading resources is “[Resource] Breaking circular references on save” but it also does not use dictionaries.

Code changes

Declaring the resource as a tool script

By declaring a resource itself a tool script using “@tool”, Godot lets its exported dictionaries be changed in the editor and prevents them from being shared between different resources. This does not fix the underlying engine behaviour, but it makes each resource have its own dictionary so editing one won’t affect the others.

This change has been applied to the resource used in the MRP and the updated MRP has been pushed to a dedicated branch for reference: [mrp-shared-dictionary](#)

By applying this change to the resource used in the MRP, each loaded resource maintains its own separate dictionary in memory. As a result, pressing one button only updates the intended resource and the shared-dictionary behavior no longer occurs. This can be viewed in the image below:

```
Godot Engine v4.7.dev.custom_build (c) 2007-present Juan Linietsky, Ariel Manzur & Godot Contributors.  
--- Debug adapter server started on port 6006 ---  
--- GDScript language server started on port 6005 ---  
Resource A: [3871368050]  
Resource B: []  
Resource A: [3871368050]  
Resource B: [3470821455]
```

Figure 1: Output log in Godot Engine when declaring the resource as a tool and adding random entries to dictionaries in resources A and B.

Patch

The Patch can be observed on: <https://github.com/Soffan-Group-6/godot-4.4.1-stable/pull/3>

- core/object/class_db.cpp In `class_get_default_property_value()`, duplicate Dictionary/Array before returning cached native class defaults so callers receive independent copies.
- modules/gdscript/gdscript.cpp In `get_property_default_value()`, duplicate Dictionary/Array when returning from `member_default_values_cache`. In `_update_exports_values()`, duplicate when passing values to editor placeholders.
- modules/gdscript/gdscript_compiler.cpp In `_parse_expression()`, skip the constant pool optimization for Dictionary/Array when `p_initializer=true`, so each instance gets a freshly constructed container instead of a shared reference. In `_prepare_compilation()`, duplicate container types before storing in `member_default_values`.
- modules/mono/csharp_script.cpp Apply the same fixes to the C# script paths: `get_property_default_value()` and `_update_exports_values()`.
- tests/ Three new GDScript test cases verifying empty containers, non-empty initializers, and `@export Resource` scenarios all produce independent instances.

Considered for acceptance (passes all automated checks)

fix: <https://github.com/Soffan-Group-6/godot-4.4.1-stable/actions/runs/22633960184>

tests: <https://github.com/Soffan-Group-6/godot-4.4.1-stable/actions/runs/22635952258>

Test results

Initial results

CI-tests:

All branches, including master, had the issue of a CI-job failing because of not being able to locate the .NET assemblies when compiling on Linux. Because of that this issue might interfere with the task, due to validation on Linux through CI not being possible, a new issue was created. Version 4.4.1-stable also failed CI jobs on macOS and Windows, including failing to compile. The Windows errors were tested manually and it was confirmed that the issue lied in the CI implementation and not the code relevant to the issue. The macOS errors could not be tested manually due to no group member having it available.

master: <https://github.com/Soffan-Group-6/godot-4.4.1-stable/actions/runs/22483267244>

4.4: <https://github.com/Soffan-Group-6/godot-4.4.1-stable/actions/runs/22486693718>

4.4.1-stable: <https://github.com/Soffan-Group-6/godot-4.4.1-stable/actions/runs/22485592458>

Unit tests:

All unit tests succeeded on master, 4.4 and 4.4.1.

master:

https://github.com/Soffan-Group-6/godot-4.4.1-stable/blob/report/logs/test_run_master.log

4.4:

https://github.com/Soffan-Group-6/godot-4.4.1-stable/blob/report/logs/test_run_44.log

4.4.1:

https://github.com/Soffan-Group-6/godot-4.4.1-stable/blob/report/logs/test_run_441stable.log

UML class diagram and its description

To visualize the features and code affected by the issue, a UML class diagram was created:

<https://github.com/Soffan-Group-6/godot-4.4.1-stable/blob/report/UML.drawio>

Key changes/classes affected

In relation to the issue many classes were affected when analyzing the pipeline executed within which the bug is present. This was mainly done through analyzing the stack trace but also by implementing print tests that present which methods are executed when the bug appears [2] [3] [4]. To present this clearly it is required to separate the case where the MRP is edited and run.

Edit MRP

The key initial process that happens whilst clicking “Edit” in the Project Manager is the SceneTree object processing followed by Object and EditorNode handling notifications and thereafter the EditorFileSystem scanning. Following this, the EditorNode handles the plugins by initializing them and adding them. Then, Node and SceneTree are used to add the plugin to the tree. After that, GDScriptInstance, GDScriptFunction and MethodBind perform their respective calls in relation to the plugin. Following, Node then sets up the SceneTree as specified by the project files. GDScriptInstance, GDScriptFunction and GDScriptUtilityFunctionsDefinitions then loads in GDScript based files within the MRP [1]. Lastly ResourceLoader loads in the Resource objects.

For the “Edit” selection, there is an extra process pipeline that happens as follows. Following the previous executions the same process happens again except, following Object handling notifications, EditorFileSystem also does that instead of scanning, followed by Object emitting a signal. The EditorNode then believes sources are changed and handles that followed by loading the editor layout and opening the scene from the MRP config file. This leads to SceneTree being set again through Node and Object and VBoxContainer handling notifications. GDScriptInstance, GDScriptFunction and GDScriptUtilityFunctionsDefinitions then loads in GDScript based files within the MRP again [1]. Similar to the previously described pipeline, lastly ResourceLoader loads in the Resource objects.

Run MRP:

When running the MRP from the Project Manager instead, the pipeline is performed. Firstly the SceneTree is set with the help of Node, Object and VBoxContainer. Then the last two steps of the previously described pipeline is performed, meaning GDScriptInstance, GDScriptFunction and GDScriptUtilityFunctionsDefinitions loading in GDScript based files within the MRP and ResourceLoader loading in Resource objects [1].

Both of the pipelines described do then provide different results in terms of how the system handles the Dictionary objects within the extended Resource object provided by the MRP [1].

Architectural overview

The architectural overview is somewhat described in the previous section, but it is of relevance to describe the relations and purpose of the system thoroughly as presented in the UML class diagram.

The parts of the system, relevant to the issue at hand, has, for the purpose of understanding the bug, been branched out with its start in the MRP class PluginMain which contains the logic related to the plugin functionality [1]. This is where the Button and Resource based objects are defined and where the logic of adding keys to the Dictionary objects and printing it out is located. This means that PluginMain uses extensions of Button and Resource, where the Resource extension MyResource contains attributes of type Dictionary.

When the project containing the plugin is initially loaded, GDScript is used for the GDScript based files whilst ResourceLoader is used to load the resource files but the pipeline goes much deeper for both of the mentioned tasks, whereas load-functions from different classes are used within the ResourceLoader and by the EditorNode depending on what is to be loaded, which means load-functions from mainly ResourceFormatText, ResourceFormatLoaderGDScript and GDScript.

PluginMain itself is also an extension of VBoxContainer which is an extension of BoxContainer which is an extension of Container which is an extension of Control. Control is then what extends CanvasItem, which extends Node, which extends Object. Since Button is an extension of BaseButton, it also ends up being an indirect extension of Control. Similar to CanvasItem, EditorNode is also an extension of Node and used many of the already mentioned classes such as Dictionary, ResourceLoader and CanvasItem within the process where the bug appears.

Moving on from the architecture, the purpose of this part of the system is to correctly load and handle data provided from a project containing a plugin with resources specifically containing Dictionary objects. As described in the previous section, the core of the responsibility of that process lies on EditorNode since the editor handles the SceneTree representing the plugin as well as its resources. Though this does not apply to the “run” process since nothing is to be edited at that point. This means it performs less of the process required for the editor where the focus is on loading in the GDScripts and Resources almost directly since all of the editor setup processes are not needed.

Relation to design pattern(s)

Since the issue at hand was in the form of a bug, it was not relevant to modify any source code until the bug was pinpointed. Since GoDot is a large project containing many classes and files, at the same time as the issue not requesting refactoring or touching code that is not the bug. Because of this design patterns were not to be used even though it would be helpful in the long run.

To involve design patterns it is relevant to present how design patterns have already been implemented in relation to the bug. Since the core of the bug lies in how resources are handled when a plugin is loaded in. The first pattern is a factory pattern since the user interface is created through preloads and dynamic initialization where the objects are separated from usage. It works like a factory creating instances of the plugin scene. Secondly template method pattern is used where a base class is defining the structure from a plugin template, which would look similar to the MRP [1]. In this case the base class would be EditorPlugin based on the architecture and the MRP represents a plugin template by using extension classes based on the source. Lastly a facade pattern is used by a much simpler API that enables easy use while interacting with the editor. This is because the editor makes it possible to build the user interface through the main screen instead of modifying internal nodes and files. For the case of the issue on the other hand, this design pattern halts the debug possibilities due to the editor stopping the processes and taking over while running. If the editor could be executed automatically and reacting to input from unit tests, this would not be an issue. But since it is difficult and no previous unit tests were implemented, it leaves a much more difficult task in terms of finding bugs within the editor.

Overall experience

Assessment of team using Essence standard

Our team is currently between formed and collaborating. We somewhat understand our responsibilities and have met virtually to coordinate work, even though individual responsibilities not was agreed upon before hand. Instead, much of the work has continued in parallel, with members exploring tasks and sharing findings as we go. One reason behind this was that the issue we are addressing is somewhat complex, requiring experimentation before being able to structure up subtasks.

To reach the next state, main obstacles is the complexity of the issue and limited prior experience with the Godot engine. Even though the issue on the surface seemed somewhat straight forward, much research had to be done in order to understand how the issue was caused, how to fix it and how to provide unit tests for the new functionality. Additionally, since individual roles were not clearly defined at the start, it took extra effort to get everyone on the same page regarding what have to be done and how to combine everyone's work.

What are your main take-aways from this project? What did you learn?

We have experienced how a non-trivial real-world issue can be: Even with a MRP, finding the source of the bug required searching through multiple engine modules rather than one obvious function, especially in quite a large open-source project [1].

It is also crucial to obtain tests and requirements when working on someone else's large codebase where we weren't involved, which will narrow down the scope (despite still being wide) for us to find the source; Moreover, having a small issue that might be rooted from the start of the development only has an exponential effect to the maintenance of the project.

In addition, the tooling and documentation quality affects productivity and the degree of maintenance after development, since it will take others or successors a large amount of time to reinvestigate the issue.

More importantly, we know that even a partial resolution with good documentation is still constructive to the project since it gives more context to others in the future who attempt to fix the issue, and is definitely helpful for the outcome of the development atmosphere.

How did you grow as a team, using the Essence standard to evaluate yourself?

We moved from a loosely formed team towards a more collaborating state by aligning on shared products (requirements list, UML diagram, test logs, branches,...) and by making our findings visible through issues and documentation rather than on in ad-hoc chats. This also improved our way of working, instead of purely exploratory debugging we started and increasingly followed a recognizable practice: defining requirements, designing tests, running regression tests and attempting changes, which brought us closer to a stable, repeatable way of working even though the technical problem remained challenging.

How would you put your work in context with best software engineering practice?

Our work followed a test-driven approach at system level: clarify requirements, analyze existing tests, add tests for bug scenarios and only then attempt the fix, which is aligned with modern SE emphasis on regression safety and automated testing.

We also applied good open-source contribution practices by reading contribution guidelines, understanding the convention of the project and respecting branching and PR rules, in addition, to avoid creating merge commits on forks and preparing clean patches.

Where we deviated from “ideal” practice was mainly delayed scoping, we underestimated how deep the bug was and only realized the complexity when we were already running out of time, we should have down-scope earlier or for the sake of the assignment and satisfaction of the team, negotiate a smaller, clearly bounded issue to work on.

Statement of contribution

Erik Olsson (erik-ol):

- Found possible issues to work on
- Debugging the issue
- Building project version with stack traces
- Unit test investigation

Chow Pun Chun (MrNoodlez-1227):

- Tested the MRP related to issue and reassured the bug
- Investigated with modified codebase of groupmates about where the issue occurs
- Understood the issue's logic and attempted to search for the source of the issue, without success
- Contributed to documentation including takeaways, growth of the team and software engineering practices

Albin Blomqvist (zzimbaa):

- Tested the MRP connected to the issue and confirmed the bug.
- Investigated the issue where dictionaries became read only when modified in the editor.
- Implemented and validated the @tool resource solution to eliminate shared dictionary behavior in the MRP.
- Investigated source code in an attempt to locate and resolve the issue, without success.
- Contributed to documentation on requirements, essence and code changes regarding declaring resources as tool scripts.

Josefine Nyholm (joss2002):

- Helped deciding on an issue to work on
- Set up working repository
- Created initial test logs based on unit tests and CI
 - Used test logs to verify that the issue could be directly worked on and documented it
- Built UML class diagram to map out the classes, attributes and methods closely related to the issue
- Performed thorough print tests on functions/methods closely related to the program sequence where the bug is present in order to find the bug
- Suggested project plan options and approaches of how to work on the chosen issue
- Sorted out distribution of information and discussions for the group as well as providing a base to work from in terms of bringing up concerns, solutions and approaches in regard to the assignment and chosen issue
- Documented "Onboarding experience"
- Documented "Overview of issue(s) and work done"
- Documented "Test results"
- Documented "UML class diagram and its description"

Avid “HotFazz” Fayaz:

- Performed deep root cause analysis of the bug by tracing through the GDScript compiler (gdscript_compiler.cpp), VM (gdscript_vm.cpp), bytecode generator (gdscript_byte_codegen.h), and analyzer (gdscript_analyzer.cpp) to identify the exact mechanism causing shared Dictionary/Array instances
- Identified the root cause: the GDScript compiler’s `_parse_expression()` treats empty Dictionary/Array literals as constants when used as member field initializers, placing them in a shared constant pool so every instance references the same object in memory
- Implemented the fix across 4 files covering 7 code paths:
 - gdscript_compiler.cpp: Skip constant optimization for Dictionary/Array when compiling member initializers; duplicate container defaults in `member_default_values`
 - gdscript.cpp: Duplicate Dictionary/Array in `get_property_default_value()` and `_update_exports_values`
 - class_db.cpp: Duplicate container types in `class_get_default_property_value()`
 - csharp_script.cpp: Same fixes for C# script paths (`get_property_default_value()` and `_update_exports_values()`)
- Wrote three GDScript test cases verifying the fix:
 - `member_empty_containers_not_shared`: empty dict/array members (typed and untyped) are independent per instance and not read-only
 - `member_container_initializers_not_shared`: non-empty constant initializers like `{"a": 1}` are not shared via the constant pool
 - `exported_containers_not_shared_across_resources`: `@export` dict/array on Resource subclasses are independent, matching the exact upstream MRP scenario
- Verified that the fix covers all identified code paths where sharing could occur, including runtime, editor/tools, and C# paths
- Contributed to report documentation

References

- [1] Minimal Reproduction Project:
https://github.com/user-attachments/files/20755517/dictionary_bug_mrp.zip
- [2] Print test branch:
- [3] Edit stack trace:
https://github.com/Soffan-Group-6/godot-4.4.1-stable/blob/report/logs/edit_stack_trace.txt
- [4] Run stack trace:
https://github.com/Soffan-Group-6/godot-4.4.1-stable/blob/report/logs/run_stack_trace.txt